Week 02
Bellman Equations and Dynamic Programming

STUDY-RL — a self-paced Deep RL curriculum

October 6, 2026

Contents

Learning objectives 3

1 Motivation & intuition 4

2 Mathematical formulation 5
- 2.1 Bellman expectation equations . . . 5
- 2.2 Bellman optimality equations . . . 5
- 2.3 The Bellman operators and contraction . . . 5
- 2.4 Generalised policy iteration . . . 6

3 Algorithms 7

4 Implementation walkthrough 8

5 Empirical results 9

6 Safety angle 11

7 Matching application 12

8 Exercises 13

9 Further reading 14

Learning objectives

Study companion. Read `teaching_guide.md` in this folder for step-by-step examples, an analogy with its limits, and self-checks. Then use `workbook.ipynb` to implement and vary the calculations.

Key Idea

By the end of this week you will be able to:

1. Write the Bellman *expectation* and *optimality* equations for V^π, $V^\star$, Q^π, $Q^\star$ from first principles.
2. Prove that the Bellman optimality operator is a γ-contraction in the sup-norm, and use Banach's fixed point theorem to deduce that $V^\star$ is the unique fixed point.
3. Implement *value iteration* and *policy iteration* in < 100 lines of NumPy on a deterministic and a stochastic GridWorld.
4. State *generalised policy iteration* (GPI) and explain why every algorithm in the rest of this course is a GPI variant.
5. Solve Jack's car rental problem (Sutton & Barto Example 4.2) to within 10^{-2} in sup-norm.
6. Articulate why DP requires a full model and what changes when we do not have one (Week 3+).

1 Motivation & intuition

Bandits gave us the irreducible exploration-exploitation problem; MDPs gave us state and transitions. Now we ask: *if I knew the MDP exactly — every transition probability, every reward — what would the optimal policy be?*

This is the dynamic-programming (DP) regime. It feels academic — in production we never know the model. But every learning algorithm in the following twenty weeks is, structurally, an approximation of one of two DP procedures: *policy evaluation* (compute V^π) and *policy improvement* (pick $\pi' \geq \pi$). When you understand the exact arithmetic of DP, the noisy stochastic-gradient version of it becomes *intelligible*, not magical.

Bellman called dynamic programming "*dynamic* programming" because "planning" was politically vague at RAND in 1953 — he needed a name no one could object to.

Intuition. Dynamic programming is what Q-learning would be if the world cooperated by telling you the full transition kernel for free. Q-learning is what DP would be if you replaced the expectation by a Monte-Carlo sample. SARSA is the same swap, on-policy. PPO is the same swap, with neural-net function approximation and a trust region. DP is the mother algorithm.

Common pitfall

"Why bother with DP if I'll never use it?" Because contraction is a reusable proof tool. With function approximation, the tabular sup-norm argument need not survive; particular projected operators retain contraction under additional assumptions (Tsitsiklis & Van Roy 1997), while other settings can diverge. Understanding which property is preserved and which is lost is how you debug a DQN.

2 Mathematical formulation

2.1 Bellman expectation equations

Fix a policy π (possibly stochastic). Define

$$V^\pi(s) \triangleq \mathbb{E}_\pi[G_t \mid S_t = s], \qquad Q^\pi(s,a) \triangleq \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a].$$

Theorem 2.1.1 (Bellman expectation equations)**.** *For every $(s,a) \in \mathcal{S} \times \mathcal{A}$,*

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V^\pi(s')\big], \tag{2.1}$$

$$Q^\pi(s,a) = \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma \sum_{a'} \pi(a' \mid s') Q^\pi(s', a')\big]. \tag{2.2}$$

Step-by-step. Expand the return: $G_t = R_{t+1} + \gamma G_{t+1}$. Hence

$$V^\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s].$$

By the law of total expectation,

$$\mathbb{E}_\pi[G_{t+1} \mid S_t = s] = \mathbb{E}_\pi[\mathbb{E}_\pi[G_{t+1} \mid S_{t+1}] \mid S_t = s] = \mathbb{E}_\pi[V^\pi(S_{t+1}) \mid S_t = s].$$

The outer expectation over (A_t, S_{t+1}, R_{t+1}) given $S_t = s$ factorises as $\pi(a \mid s)\, p(s', r \mid s, a)$, giving (2.1). The Q-case follows by the same expansion conditioned on (s,a). □

Matrix form. For finite $\mathcal{S}$ and a fixed π, define $r^\pi \in \mathbb{R}^{|\mathcal{S}|}$ with $r^\pi(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) r$ and $P^\pi \in \mathbb{R}^{|\mathcal{S}| \times |\mathcal{S}|}$ with $P^\pi(s,s') = \sum_a \pi(a \mid s) \sum_r p(s', r \mid s, a)$. Then (2.1) reads

$$V^\pi = r^\pi + \gamma P^\pi V^\pi \quad \Longleftrightarrow \quad V^\pi = (I - \gamma P^\pi)^{-1} r^\pi, \tag{2.3}$$

which exists because $\|\gamma P^\pi\|_\infty \le \gamma < 1$, so $I - \gamma P^\pi$ is strictly diagonally dominant and invertible. This is *policy evaluation as a linear system.*

2.2 Bellman optimality equations

Define $V^\star(s) = \max_\pi V^\pi(s)$ and similarly $Q^\star$. A greedy step over $Q^\star$ gives

$$V^\star(s) = \max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V^\star(s')\big]. \tag{2.4}$$

2.3 The Bellman operators and contraction

Define two operators on $\mathbb{R}^{|\mathcal{S}|}$:

$$(T_\pi V)(s) \triangleq \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)[r + \gamma V(s')], \tag{2.5}$$

$$(T_\star V)(s) \triangleq \max_a \sum_{s',r} p(s', r \mid s, a)[r + \gamma V(s')]. \tag{2.6}$$

Theorem 2.3.1 (γ-contraction in sup norm)**.** *For all $V, W \in \mathbb{R}^{|\mathcal{S}|}$, $\|T_\pi V - T_\pi W\|_\infty \le \gamma \|V - W\|_\infty$ and $\|T_\star V - T_\star W\|_\infty \le \gamma \|V - W\|_\infty$.*

Full derivation. For T_π: subtract the operator applied to two arguments and pull the absolute value through the convex combination:

$$\begin{aligned} |(T_\pi V)(s) - (T_\pi W)(s)| &= \left| \sum_{a,s',r} \pi(a \mid s) p(s', r \mid s, a) \gamma [V(s') - W(s')] \right| \\ &\le \gamma \sum_{a,s'} \pi(a \mid s) P^a_{ss'} |V(s') - W(s')| \\ &\le \gamma \sum_{a,s'} \pi(a \mid s) P^a_{ss'} \|V - W\|_\infty \\ &= \gamma \|V - W\|_\infty, \end{aligned}$$

where we used $\sum_{s'} \sum_a \pi(a \mid s) P^a_{ss'} = 1$ (probabilities sum to one). Taking the supremum over s on the left gives the claim.

For $T_\star$ the only extra ingredient is the inequality $|\max_a f(a) - \max_a g(a)| \le \max_a |f(a) - g(a)|$ (because both maxes are achieved and we can compare componentwise). Apply this with $f(a) = \sum_{s',r} p(s', r \mid s, a)[r + \gamma V(s')]$ and $g(a)$ likewise for W, then carry out the same algebra as above on the inner sum. □

Corollary 2.3.2 (Banach fixed-point)**.** *The operator $T_\star$ has a unique fixed point in $\mathbb{R}^{|\mathcal{S}|}$, which equals $V^\star$. Moreover, for any initial V_0 the sequence $V_{k+1} = T_\star V_k$ converges geometrically:*

$$\|V_k - V^\star\|_\infty \le \gamma^k \|V_0 - V^\star\|_\infty.$$

This is value iteration*.*

Sketch. $\mathbb{R}^{|\mathcal{S}|}$ with the sup norm is a complete metric space. By Theorem 2.3.1, $T_\star$ is a γ-contraction with $\gamma < 1$, so Banach's theorem gives a unique fixed point, and the iteration $V_{k+1} = T_\star V_k$ has linear convergence at rate γ. The fixed point satisfies (2.4), hence equals $V^\star$. □

Policy improvement theorem.

Theorem 2.3.3. *Let π, π' be deterministic policies with $Q^\pi(s, \pi'(s)) \ge V^\pi(s)$ for all s. Then $V^{\pi'}(s) \ge V^\pi(s)$ for all s, with strict inequality at any state where $Q^\pi(s, \pi'(s)) > V^\pi(s)$. Changing between tied actions need not change the value.*

Idea. Iterate the assumed inequality $V^\pi(s) \le Q^\pi(s, \pi'(s)) = \mathbb{E}[R_{t+1} + \gamma V^\pi(S_{t+1}) \mid S_t = s, A_t = \pi'(s)]$. Unrolling V^π on the right side and re-applying the inequality at each step telescopes to $V^\pi(s) \le V^{\pi'}(s)$. A strict initial advantage yields strict improvement at that state; merely changing an action label does not. □

2.4 Generalised policy iteration

GPI is the umbrella name for the interaction *Evaluation* → *Improvement*, repeated until both stabilise. The two "arrows" need not be exact:

- Value iteration does *one* step of evaluation (pulling a max on each sweep) and then improves implicitly.
- Policy iteration does *full* evaluation, then a single improvement step.
- Modified policy iteration (truncated PI) does k steps of evaluation between improvements — and we can prove convergence for any $k \ge 1$.

GPI is a useful organising view of many value-based and actor–critic algorithms in this course; it is not a literal description of every method.

3 Algorithms

Algorithm 1: In-place (Gauss–Seidel) value iteration.

Input: discount γ, tolerance ϵ.

Initialise $V(s) \leftarrow 0$ for all s.;
repeat
  $\Delta \leftarrow 0$;
  foreach $s \in \mathcal{S}$ **do**
    $v \leftarrow V(s)$; $V(s) \leftarrow \max_a \sum_{s',r} p(s', r \mid s, a)[r + \gamma V(s')]$;
    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$;
  end
until $\Delta < \epsilon$;
Output greedy policy: $\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s', r \mid s, a)[r + \gamma V(s')]$;

Algorithm 2: Howard's policy iteration.

Input: discount γ.

Initialise π arbitrarily.;
repeat
  /* (i) Policy evaluation: solve $(I - \gamma P^\pi)V = r^\pi$ or iterate until $\|V_{k+1} - V_k\|_\infty < \epsilon$. */
  Compute V^π.;
  /* (ii) Policy improvement. */
  foreach $s \in \mathcal{S}$ **do**
    $\pi^{\text{new}}(s) \leftarrow \arg\max_a \sum_{s',r} p(s', r \mid s, a)[r + \gamma V^\pi(s')]$;
  end
  Set stable $\leftarrow \pi^{\text{new}} = \pi$; $\pi \leftarrow \pi^{\text{new}}$;
until *policy stable*;

Complexity. One VI sweep costs $O(|\mathcal{S}|\,|\mathcal{A}|\,|\mathcal{S}|)$. PI does $O(|\mathcal{S}|^3)$ for the linear solve plus $O(|\mathcal{S}|^2|\mathcal{A}|)$ for improvement; in practice PI converges in $O(\log(1/\epsilon))$ outer iterations while VI needs $O(\log(1/\epsilon)/\log(1/\gamma))$. PI is therefore favoured when γ is close to one; VI when $|\mathcal{S}|$ is large.

Example 3.0.1 (A 3-state worked example)**.** Consider three states $\{1, 2, 3\}$, one action that always transitions $s \to s+1$ deterministically with reward 0 except $3 \to 3$ with reward 1 (absorbing). Set $\gamma = 0.9$. Then $V^\star(3) = 1 + 0.9V^\star(3)$ gives $V^\star(3) = 10$; $V^\star(2) = 0.9 \cdot 10 = 9$; $V^\star(1) = 0.9 \cdot 9 = 8.1$. Run value iteration starting at $V_0 = 0$: $V_1 = (0, 0, 1)$, $V_2 = (0, 0.9, 1.9)$, $V_3 = (0.81, 1.71, 2.71)$, …, converging geometrically.

Common pitfall

Stochastic environments. If you forget that policy iteration *must* use a stochastic transition table (not just the deterministic "most-likely next state"), you compute a policy that is greedy w.r.t. the wrong dynamics. The slipping GridWorld in the workbook reveals this bug loudly: the policy looks fine in the deterministic case and falls into the pit in the slipping case.

4 Implementation walkthrough

The shared utility `common.tabular.value_iteration` does the full DP loop; the project layer adds Jack's car rental MDP.

```python
# common/tabular.py (excerpt)
def value_iteration(env, gamma=0.99, tol=1e-8, max_iter=10_000):
    P = env.transition_table()
    V = np.zeros(env.nS); deltas = []
    for _ in range(max_iter):
        V_new = np.zeros_like(V)
        for s in range(env.nS):
            qs = np.full(env.nA, -np.inf)
            for a in range(env.nA):
                qs[a] = sum(p * (r + gamma * V[ns] * (not d))
                            for p, ns, r, d in P[(s, a)])
            V_new[s] = qs.max()
        delta = np.max(np.abs(V_new - V))
        deltas.append(float(delta))
        V = V_new
        if delta < tol: break
    return V, ..., deltas
```

Jack's car rental builds a full transition kernel *analytically*: rentals are Poisson, returns are Poisson, the agent moves a cars overnight, and we marginalise out both Poisson variables to compute the expected reward and the distribution over tomorrow's stock per location. The per-location calculation is independent given the action, which makes the implementation $O(K^2 P)$ in time where K is the capacity and P is the Poisson truncation point.

```python
# project/carrental.py (excerpt)
def expected_reward_and_next(self, s, a):
    n1, n2 = s
    a_eff = max(-n2, min(n1, a))           # feasible move
    n1m, n2m = n1 - a_eff, n2 + a_eff
    move_r = -self.move_cost * abs(a_eff)
    rent_r = self.rental_reward * (self.exp_req[0][n1m] + self.exp_req[1][n2m])
    joint = np.outer(self.next_dist[0][n1m], self.next_dist[1][n2m]).ravel()
    return move_r + rent_r, joint
```

5 Empirical results

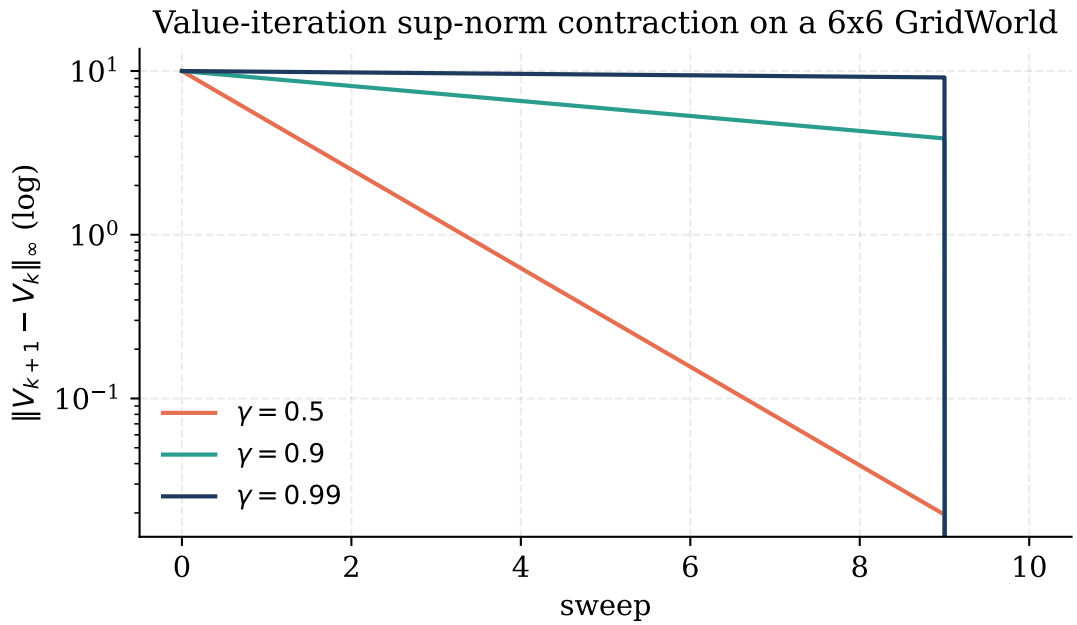


Figure 5.1: Sup-norm gap $\|V_{k+1} - V_k\|_\infty$ during value iteration on a 6×6 GridWorld, for three discount factors. The slope on the semi-log axis equals $\log \gamma$, exactly as Theorem 2.3.1 predicts.

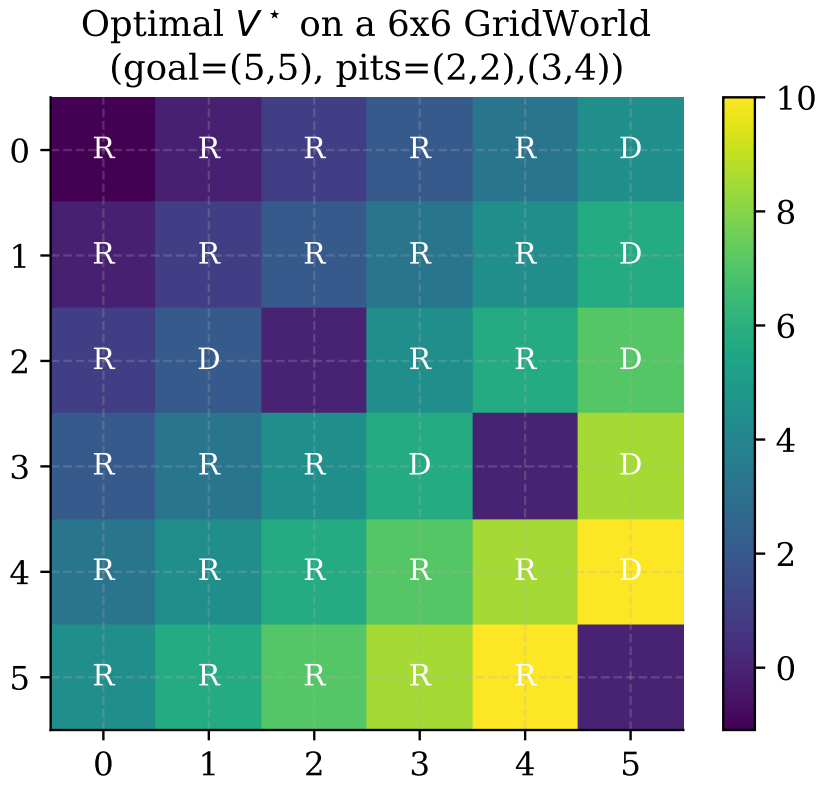


Figure 5.2: Optimal value function on a 6×6 GridWorld with two pits. The greedy arrows route around the pits; the values fall off geometrically with distance to the goal.

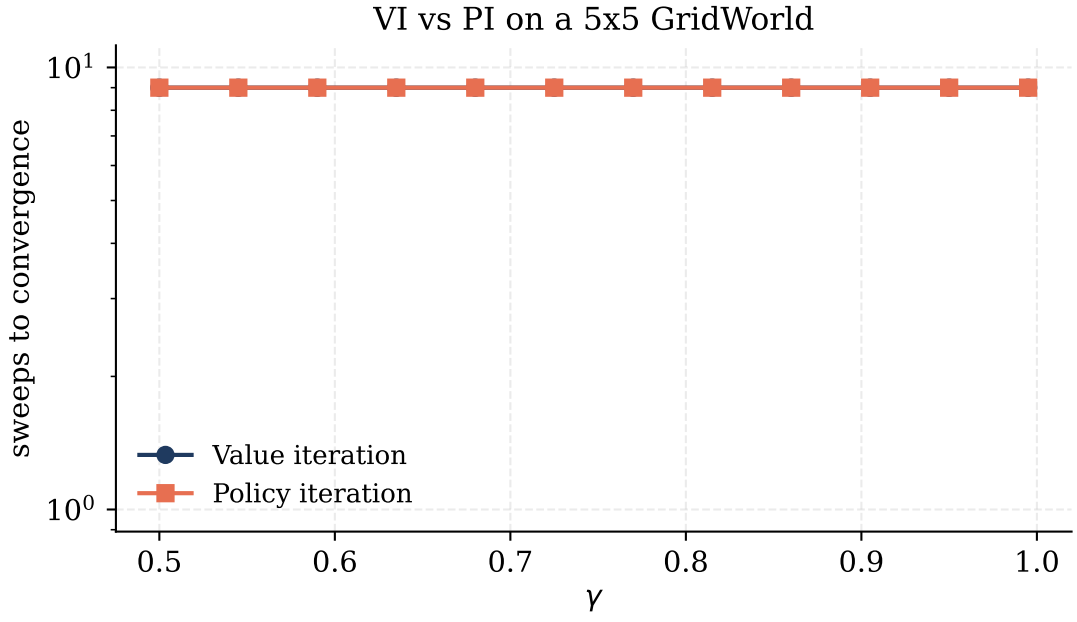


Figure 5.3: Sweeps to convergence for VI vs PI as γ varies. PI is nearly flat in γ (it stabilises in a small number of outer iterations); VI diverges like $1/(1-\gamma)$ as $\gamma \to 1$.

6 Safety angle

Safety angle

Planning with the wrong model. DP gives the optimal policy *conditional on the model being correct*. In safety-critical settings two failure modes dominate:

1. *Misspecified* $p(s' \mid s, a)$. If your simulator under-models ice on roads or under-loads a hospital ICU, the planner is happy and the deployment kills people. Robust DP (Iyengar 2005; Nilim & El Ghaoui 2005) replaces the inner max with a worst-case over a set of plausible kernels: $V(s) \leftarrow \max_a \min_{p \in \mathcal{P}_a} \mathbb{E}_p[r + \gamma V(s')]$. The result is conservative; the regret bound is in terms of the "ambiguity set" diameter.
2. *Reward hacking.* If the reward is a noisy proxy of what you actually want (binding $\neq$ value, hire $\neq$ retained-hire), the DP-optimal policy is the proxy-optimal policy, and the proxy and actual goals diverge. See Anthropic's Natural Emergent Misalignment from Reward Hacking (arXiv:2511.18397, 2025) for empirical evidence that this is *not* only a theoretical worry.

Mitigations. Use "constrained MDPs" (Week 22) when there is an auxiliary safety cost; use robust DP when the model is uncertain; use shielding (Week 23) when there are hard constraints that no value-function fudge can be trusted to respect.

7 Matching application

Matching application

synthetic insurance as an MDP. In Week 1 we treated lead-broker routing as a contextual bandit. Strictly that is correct only if the action does not change future state. In practice:

- Brokers have queues. Saturating a star broker reduces their *tomorrow* capacity — the bandit reward of routing to them is therefore part of an MDP whose state includes the broker queue.
- Negotiation. A broker who declines today is less likely to be offered a similar lead tomorrow — there is a softer state in their "relationship score" with the platform.

Both are captured by an MDP whose state augments the lead context with a slow-moving broker capacity vector. With finite states this MDP could be solved by exact DP — it is the example par excellence to motivate *learning* when the model is unknown.

ATS as an MDP. A multi-stage hiring funnel (screening → interviewing → offer → acceptance) is naturally a finite-horizon MDP. Each stage's policy — "advance to next stage" or "reject" — can be optimised by backward induction. The Bellman equation reads $V_t(s) = \max_{a \in \{\text{advance, reject}\}} \mathbb{E}[r_t + V_{t+1}(s') \mid s, a]$. Week 14 (offline RL) returns to this with conservative Q-learning on logs of past hiring decisions.

8 Exercises

See `workbook.ipynb`. Highlights:

Ex. 1 ($\star$). Apply the Bellman optimality operator to a zero value function on a 3×3 grid.

Ex. 2 ($\star$). Run value iteration on a 5×5 deterministic GridWorld and confirm geometric convergence at rate γ.

Ex. 3 ($\star\star$). Policy iteration on the same grid; check $V^{\mathrm{PI}} = V^{\mathrm{VI}}$ to floating-point precision.

Ex. 4 ($\star\star$). Stochastic GridWorld with slip $\rho \in \{0, 0.2, 0.4\}$. Plot $V^\star(s_0)$ as a function of ρ.

Ex. 5 ($\star\star$). Verify the sup-norm contraction bound empirically.

Ex. 6 ($\star\star$). Solve policy evaluation via the linear system $(I - \gamma P^\pi)V = r^\pi$ and compare with iterative evaluation.

Ex. 7 ($\star\star\star$, open-ended). Implement *modified policy iteration* (truncated GPI). For which k does the schedule become VI? PI?

Ex. 8 ($\star\star\star$, applied). Jack's car rental (Sutton & Barto Example 4.2) with cap = 5.

9 Further reading

The companion `extended_reading.pdf` unpacks Bellman (1957), Howard (1960), Puterman (1994 Ch. 6–7), Tsitsiklis & Van Roy (1997 — divergence of off-policy DP with linear FA), and Munos & Szepesvári (2008) on finite-sample analysis of approximate dynamic programming. Source material in `research/deeprl_core_sota.md` (§1) and Sutton & Barto Chapter 4.